iOS Terminal Connection Guide - Apple Tap to Pay

The **iOS Terminal Connection Guide** provides a comprehensive walkthrough for enabling and integrating Apple Tap to Pay on iPhone within your iOS application. This guide will cover the following areas:

- **Entitlement Setup**: How to apply for and configure the required Apple entitlement to enable Tap to Pay functionality.
- **Environment Configuration**: Instructions for connecting to Apple's production and certification environments for development and testing.
- **Connecting Apple Tap to Pay Terminal Device**: Steps to initiate the connection, monitor progress, and handle success or failure through delegate methods.
- **Response Handling**: Guidance on interpreting connection statuses and resolving common initialization or pairing issues.

Prerequisites

Before you begin integrating the Deluxe POS SDKs, ensure you have the following prerequisites ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and retrieve associated merchant accounts, which are necessary to establish a connection with the terminal.
  - iOS Merchant Authorization & Accounts Guide

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

ON THIS PAGE

Prerequisites

Setup Requirement

Entitlement Setup

Environment Configuration

Connecting Apple Tap to Pay Terminal Device

Response Handling

functionalities and uses a TerminalManagerDelegate protocol object to asynchronously provide updates on various connection statuses throughout the connection process.

After integrating all the .xcframework files into your project, you'll have access to all the classes provided by the POSSDK framework. To use any of these classes, simply import the POSSDK module into your own class.

```
import PosSDK
```

Entitlement Setup

How to apply for and configure the required Apple entitlement to enable Tap to Pay functionality.

Before proceeding, ensure that:

- You are enrolled in the Apple Developer Program.
- You have full access to the necessary resources in the Apple Developer Portal.

If you haven't enrolled yet, visit the https://developer.apple.com/programs/enroll/ page to get started.

Complete the Following Steps

To proceed, make sure you've completed these two key setup tasks:

- **Obtain Apple Approval:** Request and receive approval for the iOS entitlement required to enable Tap to Pay on iPhone.
- **Establish Device Connectivity:** Set up and verify connectivity between your device and the appropriate Apple environments.

Requesting the iOS Entitlement

To enable Tap to Pay on iPhone, you must complete the entitlement application through the Apple Developer Portal. This process requires an Organization-level Apple Developer account, and only the Account Holder can access and submit the entitlement request form.

Once your request is approved, update your project's entitlement file with the following key-value pair:

```
<key>com.apple.developer.proximity-
reader.payment.acceptance</key>
<true/>
```

For complete setup instructions, please refer to Apple's official documentation: https://developer.apple.com/documentation/proximityreader/setting-up-the-entitlement-for-tap-to-pay-on-iphone.

Environment Configuration

Instructions for connecting to Apple's production and certification environments for development and testing.

Establish Device Connectivity with Apple Server Environments

Your application must connect to two distinct Apple Tap to Pay server environments:

Apple Environment	Description
Certification	A test environment used to validate production-level code with test data. Only test cards should be used in this environment to ensure compliance.
Production	The live environment for processing real transactions with production data.

Note: *If you configure the POSService singleton with production environment, then it will be pointing to Apple production environment otherwise it will be pointing to Apple certification environment. By default the POSService singleton object is configured with validation environment.*

Device Connectivity with Apple Server Environments

Your iOS device must be able to connect to both Apple's Production and Certification environments, depending on the stage of development.

Production Environment

default. No additional configuration is necessary.

Certification Environment

Required for development and testing

To test Tap to Pay features, your device must connect to Apple's Certification environment. There are two supported methods:

Option 1: Use the Proximity Reader Certification Configuration Profile

Available for devices running iOS 18.4 or later.

Download and install the Proximity Reader Certification Configuration Profile on your iOS device.

To install:

Go to Settings > Profile Downloaded, then tap Install (top right).

This profile auto-expires after 30 days.

Option 2: Use a Sandbox Tester Account. Applicable for all iOS versions.

Applicable to all iOS versions.

Create a *new* Apple ID to use as a dedicated sandbox Apple Developer account.

Note: *If the Apple ID is already associated with a Developer Program, it cannot be used as a sandbox account.*

Follow Apple's instructions to Create a Sandbox Apple Account .

Once created, sign in to your test iPhone with this account to perform test transactions.

Connecting Apple Tap to Pay Terminal Device

Step 1: Prepare Merchant, Terminal, and POS Application Information

Begin by defining the MerchantInformation and DeviceInformation objects. These are required to initialize the TerminalManager and

```
MerchantInformation(
companyId: "",// This must be set to "000005".
mid: "",// This should be mid of your merchant
account.
tid: "",// This Should be tid of your mechant
account.
bankId: "",// This can be left empty as of now.
merchantDBA: "",// This should be dba of your
merchant account.
accountId: "", // This should be accessToken of the
merchant account.
routing : "",
authClientAccessToken: "" // This should be your
client application login bearer token that retrieved
using the MerchantAuthorizationService.
merchantAccessToken: "" // This should be accessToken
of the merchant account.
```

The authClientAccessToken can be retrieved using your MerchatAuthorizationService class and merchantAccessToken can be retrieved by using the MerchantAccountService class.

Terminal Information

Terminal information details for Apple Tap to Pay.

```
class TerminalInformation(
manufacturer = TerminalManufacturer.Apple,
model = TerminalModel.appleTapToPay,
connectionType =
TerminalConnectionType.notApplicable,
terminalAppName = "DlxCrtiOS",//You can keep this
value now
terminalAppVersion =  "0001",//You can keep this
value now
terminalName = "", // This can be left empty as its
not applicable in the case of Apple TapToPay
firmwareInfo = "",// This can be left empty as its
not applicable in the case of Apple TapToPay
deviceID = ""// This can be left empty as its not
applicable in the case of Apple TapToPay
batteryPercentage = ""// This can be left empty as
its not applicable in the case of Apple TapToPay
```

POSApp Information

```
class POSAppInformation {
name = "" //This will be the client  application name
version = "" //This will be the client  application
version
}
```

Step 2: Create and Initialize Terminal Manager object

Create TerminalManager object.

```
merchantPosAppInformation)
```

```
terminalManager?.initializeTerminalManager(delegate:
self)
```

Step 3: Check Merchant License Status

The checkMerchantLicenceStatus(delegate:) method is used to verify whether the merchant has accepted the Apple TapToPay Terms and Conditions (license agreement)

```
terminalManager?.checkMerchantLicenceStatus(delegate:
self)
```

delegate: The class instance that conforms to MerchantLicenceDelegate. It receives callback responses for the check result.

```
extension ConnectionViewController:
MerchantLicenceDelegate {

func OnCheckLicenseStatusAvailable(status:
LicenceStatus) {

}
func OnCheckLicenseStatusFalilure(status:
LicenceStatus, errorMessage: String) {

}
}
```

Success Callback:

OnCheckLicenseStatusAvailable(status: LicenceStatus) Called when the license status check is successfully performed.

status: Indicates the current phase of the Apple Tap to Pay license status.

- **pending**: The merchant has not yet accepted the Terms and Conditions. You may prompt the user to review and accept them.
- **accepted**: The merchant has successfully accepted the Terms and Conditions.
- **notApplicable**: This status indicates that the Apple Tap to Pay license process is not relevant or required for the selected terminal.

errorMessage: String) Called when the SDK is unable to complete the license status check due to an error.

status: Indicates the current phase of the Apple Tap to Pay license status.

- **unknown**: The license status could not be determined. Possibly due to internal issue.
- **failureDueToNoInternet**: The license check failed due to no internet connectivity. Retry once connectivity is restored.

errorMessage: A string that provides additional context about the current status. In some cases, this may be a plain message (e.g., "Check license status failed due to no internet")

Step 4: Invoke connect() to Start Connection

Call connect() to start the Apple TaptoPay connection.

```
terminalManager?.connect()
```

Step 5: Implement Terminal Connection Callback

Implement all the TerminalConnectionDelegate protocol in your class to receive various updates from the TerminalManager object.

```
extension ConnectionViewController:
TerminalConnectionDelegate {

func onConnectionFailed(connectionStatus:
PosSDK.TerminalConnectionStatus,
                   errorMessage: String) {
//Handle connection failed here
}

func onConnectionProgress(currentStatus:
PosSDK.TerminalConnectionStatus,
                     currentStatusDescription:
String) {
//Handle Connection progress here based on connection
status
}
```

Response Handling

On Progress

The onConnectionProgress(...) method is triggered multiple times during the connection process to deliver real-time updates on the

phase.

currentStatus: Indicates the current phase of the Apple Tap to Pay initialization process. Possible values include:

- **tapToPayInitializationStarted**: Initialization has begun. The system is performing initial communication and configuration routines. (**NOTE:** The first-time connection may take up to 40 seconds due to one-time setup and provisioning. Subsequent connections are significantly faster.)
- **tapToPayInitializationInProgress**: Initialization is actively underway.
- **tapToPayInitializationSucceeded**: The terminal has been successfully initialized and is ready to accept payments.

currentStatusDescription: A string providing additional context about the current status. For example: "TapToPay Initialization Started"

Note: *During the initial connection, currentStatusDescription may also include a progress percentage (0–100). This can be used to display a progress indicator in your UI to inform the merchant of the setup status.*

On Success

Once initialization is successful, Tap to Pay transitions to the connected state, and the onConnectionSucceeded() method is triggered.

On Failure

If the connection fails, the onConnectionFailed() method is triggered with the reason for the failure.

```
fun onConnectionSucceeded() {
// Terminal successfully connected and ready for
transactions
}

func onConnectionFailed(connectionStatus:
TerminalConnectionStatus, errorMessage: String) {
// Handle connection failure or cancellation
}
```

Note: *Please refer to the Complete SDK Reference for additional details iOS SDK Reference Guide*

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave, Minneapolis, MN 55402.

© 2026 Deluxe Corporation